

Farm Stays

Full-Stack Web Application — Completion Roadmap

Product: GCC farm / homestay / venue marketplace **Stack:** Next.js 14 · Supabase · Prisma · Stripe · Tailwind

Horizon: ~10–14 weeks to production MVP **Date:** 31 Jul 2026

North star: One real path — *Host lists* → *Guest searches* → *Quotes* → *Pays* → *Books* → *Host/Admin see the same record* — on a single database, with authenticated APIs. Stop adding dashboard screens until this spine works.

1. Current state (honest snapshot)

Area	Status	Gap
Guest browse / search / listing detail	STRONG UI	Catalog still mixes mock + localStorage; SSR incomplete
Host & admin dashboards	BROAD DEMO	Mostly localStorage; not shared with guest bookings
Auth (Supabase + demo mode)	PARTIAL	Client guards only; APIs under-protected
Listings persistence (Prisma)	PARTIAL	Optional flag; JSON payload; SQLite vs Postgres drift
Checkout / Stripe	STUB	No PaymentIntent; client-trusted totals
Booking lifecycle	SPLIT	API exists but unused; host UI uses separate store
Arabic / RTL i18n	INCOMPLETE	English-only routing despite bilingual fields
Tests / CI / SEO	MISSING	No automated tests; thin metadata

2. Guiding principles

- **Single source of truth** — Postgres (Supabase) for listings, availability, bookings, pricing settings, reviews.
- **Server owns money & authz** — recompute prices; never trust client totals; middleware + API role checks.
- **Thin UI over fat domain** — keep existing React screens; swap localStorage hooks for API clients.
- **Ship vertical slices** — each phase ends with a demoable end-to-end flow.

3. Phased roadmap

Phase 0 — Foundation & alignment (Week 1)

CRITICAL

Stabilize the platform before feature work.

- ❑ Align docs & env: Postgres/Supabase (drop SQLite for shared env); real prisma migrate
- ❑ Define env matrix: local demo vs shared DB vs production
- ❑ Schema plan: normalize Listing core fields; keep JSON payload only as bridge
- ❑ Auth model: roles in server-verified claims (not client-only metadata)
- ❑ API conventions: REST under `/api/*`, Zod validation, consistent errors
- ❑ Decision log: commission %, pending expiry, cancellation tiers (product rules)

Exit: Migrations run against Supabase; team agrees on booking status machine.

Phase 1 — Secure listings spine (Weeks 2–3)

CRITICAL

One catalog the world can trust.

- ❑ Auth on all listing write APIs (host owns; admin can patch)
- ❑ Migrate host listing create/edit/approve/unpublish off localStorage → Prisma
- ❑ Server-rendered public catalog (home, search, detail) from DB
- ❑ Seed script: migrate mock STAYS → DB; keep seeds for empty envs only
- ❑ Availability calendar table wired to host calendar UI
- ❑ Country required for search (already started) enforced server-side

Exit: Host creates listing → admin approves → guest sees it on another device/browser.

Phase 2 — Pricing & quote engine on server (Week 4)

CRITICAL

- ❑ Persist pricing settings, seasons, rooms, extras in DB
- ❑ Move `calculateStayQuote` to shared server module; expose `POST /api/quotes`
- ❑ Listing detail & checkout always quote from server
- ❑ Extras / experiences selection → quote line items (IDs only from client)
- ❑ Tax / service fee / platform commission from platform config table

Exit: Same dates/guests produce identical totals in UI and API tests.

Phase 3 — Checkout, Stripe & bookings (Weeks 5–7)

CRITICAL

Close the guest money funnel — highest product leverage.

- ❑ Replace checkout placeholder with real multi-step checkout
- ❑ Stripe PaymentIntent (or Checkout Session) + webhook → paymentStatus
- ❑ Authenticated POST /api/bookings; ignore client totalPrice; use server quote
- ❑ Use existing confirmBooking transaction (overlap + capacity + blocked dates)
- ❑ Status machine: pending → confirmed / declined / expired / cancelled / completed
- ❑ Host bookings dashboard reads Prisma bookings (delete localStorage twin)
- ❑ Admin bookings / financial views read same rows
- ❑ Email/SMS stubs: booking confirmed, host notified (Resend / Twilio later)

Exit: Guest pays in test mode → booking appears for host & admin → calendar blocks dates.

Phase 4 — Policies & marketplace rules (Week 8)

HIGH

- ❑ Instant book vs request-to-book (host accept/decline SLA)
- ❑ Pending booking expiry job (cron / Supabase scheduled function)
- ❑ Cancellation & refund rules → Stripe refunds where eligible
- ❑ Commission ledger entries per booking
- ❑ Guest verification gates (optional for high-value stays)

Exit: Documented policy matrix enforced in code, not only UI copy.

Phase 5 — Trust, reviews & messaging (Weeks 9–10)

HIGH

- ❑ Reviews only after completed stay; moderated by admin
- ❑ Guest ↔ host messaging thread per booking
- ❑ Disputes / support tickets tied to booking IDs (replace mock stores)
- ❑ Host verification documents stored in Supabase Storage
- ❑ Report listing / trust flags with audit trail

Exit: Post-stay review flow works; support can open a booking timeline.

Phase 6 — i18n, SEO & growth surfaces (Week 11)

MEDIUM

- Enable ar locale + messages/ar.json; verify RTL on key pages
- generateMetadata per listing; sitemap.xml; robots.txt
- OG images for listings; structured data (LodgingBusiness)
- Promotions / featured boost backed by DB + search ranking
- Multi-country + currency display (AED baseline, FX later)

Exit: Arabic homepage/search/listing usable; listing URLs indexable.

Phase 7 — Hardening & launch (Weeks 12–14)

HIGH

- Middleware: protect /host/*, /admin/*, sensitive APIs
- Rate limits on auth, quote, booking, listing mutation
- Tests: quote invariants, booking overlap, API authz (Vitest / Playwright smoke)
- Observability: error tracking (Sentry), structured logs, Stripe webhook retries
- Load test booking confirm path (extend existing k6 script)
- Staging deploy + checklist; remove demo-auth in production
- Legal: terms, privacy, cancellation disclosure pages
- Launch: soft launch with 5–10 hosts in one country

Exit: Staging pass + production soft launch readiness.

4. Suggested workstreams (parallel after Phase 1)

Backend

- Schema & migrations
- Authz middleware
- Quote & booking APIs
- Stripe webhooks
- Jobs / expiry

Frontend

- Swap data hooks → API
- Checkout UX
- Host booking inbox
- Empty / error states
- Arabic / RTL polish

5. Definition of “full-stack complete” (MVP)

#	Capability	Done when...
1	List	Host publishes; admin can approve; guest sees live inventory from DB
2	Discover	Search by country (+ filters) returns server results
3	Quote	Server quote matches UI; extras/rooms included
4	Pay	Stripe test payment succeeds; webhook confirms

#	Capability	Done when...
5	Book	Booking row + blocked dates; host & admin see it
6	Operate	Cancel/refund policy path works for at least one rule set
7	Trust	Post-stay review can be submitted once
8	Secure	Unauthenticated users cannot mutate listings/bookings

6. Explicitly defer (post-MVP)

- Native mobile apps
- Full multi-currency settlement / payouts to hosts (start with manual payouts)
- AI recommendations / dynamic pricing
- Complex CRM / marketing automation
- More admin screens beyond what the booking spine needs

7. Immediate next 5 engineering tickets

1. Postgres + Prisma migrate; remove production reliance on SQLite / checked-in `dev.db`
2. Add session auth to `POST/PATCH/DELETE /api/listings`
3. Wire public listings read path to Prisma for homepage + `/listings`
4. Build checkout page consuming `POST /api/quotes`
5. Stripe `PaymentIntent` + webhook → call `confirmBooking`